ABC / XYZ Segmentation - H&M

Este notebook realiza la segmentación ABC (por valor) y XYZ (por variabilidad) de clientes usando datos de transacciones. El objetivo es identificar los clientes más valiosos y entender la estabilidad de sus compras para estrategias personalizadas.

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, sum as spark_sum, stddev,
count, year, month
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

spark = SparkSession.builder.appName("ABC_XYZ_HYM").getOrCreate()
```

Carga y exploración inicial de datos

```
df = spark.read.parquet('../merge_pyspark')
print(f"Total de registros: {df.count():,}")
df.printSchema()
df.show(5)
```

```
Total de registros: 31,788,324
root
 |-- customer_id: string (nullable = true)
 |-- article_id: long (nullable = true)
 |-- Fecha: date (nullable = true)
 |-- price: double (nullable = true)
 |-- sales_channel_id: long (nullable = true)
 |-- product_code: integer (nullable = true)
 |-- prod_name: string (nullable = true)
 |-- product_type_no: integer (nullable = true)
 |-- product_type_name: string (nullable = true)
 |-- product_group_name: string (nullable = true)
 |-- graphical_appearance_no: integer (nullable = true)
 |-- graphical_appearance_name: string (nullable = true)
 |-- colour_group_code: integer (nullable = true)
 |-- colour_group_name: string (nullable = true)
 |-- perceived_colour_value_id: integer (nullable = true)
 |-- perceived_colour_value_name: string (nullable = true)
 |-- perceived_colour_master_id: integer (nullable = true)
 |-- perceived_colour_master_name: string (nullable = true)
 |-- department_no: integer (nullable = true)
 |-- department_name: string (nullable = true)
 |-- index_code: string (nullable = true)
```

```
 |-- index_name: string (nullable = true)
 |-- index_group_no: integer (nullable = true)
 |-- index_group_name: string (nullable = true)
 |-- section_no: integer (nullable = true)
 |-- section_name: string (nullable = true)
 |-- garment_group_no: integer (nullable = true)
 |-- garment_group_name: string (nullable = true)
 |-- detail_desc: string (nullable = true)
 |-- FN: double (nullable = true)
 |-- Active: double (nullable = true)
 |-- club_member_status: string (nullable = true)
 |-- fashion_news_frequency: string (nullable = true)
 |-- age: integer (nullable = true)
 |-- postal_code: string (nullable = true)

+--------------------+----------+----------+------------------
+----------------+------------+--------------------+--------------
+----------------+------------------+----------------------
+-------------------------+-----------------+-----------------
+-------------------------+--------------------------
+--------------------------+-----------------------------+-------------
+----------------+----------+----------------+-------------
+----------------+----------+--------------------+----------------
+------------------+--------------------+----+------
+------------------+----------------------+---+--------------------+
|         customer_id|article_id|     Fecha|             price|
sales_channel_id|product_code|           prod_name|product_type_no|
product_type_name|product_group_name|graphical_appearance_no|
graphical_appearance_name|colour_group_code|colour_group_name|
perceived_colour_value_id|perceived_colour_value_name|
perceived_colour_master_id|perceived_colour_master_name|department_no|
department_name|index_code|      index_name|index_group_no|
index_group_name|section_no|        section_name|garment_group_no|
garment_group_name|         detail_desc|  FN|Active|
club_member_status|fashion_news_frequency|age|         postal_code|
+--------------------+----------+----------+------------------
+----------------+------------+--------------------+--------------
+----------------+------------------+----------------------
+-------------------------+-----------------+-----------------
+-------------------------+--------------------------
+--------------------------+-----------------------------+-------------
+----------------+----------+----------------+-------------
+----------------+----------+--------------------+----------------
+------------------+--------------------+----+------
+------------------+----------------------+---+--------------------+
|00030bc026428965d...| 673677015|2019-11-28|0.0254067796610169|
2|      673677|      Henry polo (1)|            252|          Sweater|
Garment Upper body|               1010016|                  Solid|
73|        Dark Blue|                       4|
```

```
Dark|                          2|                          Blue|
1616|        Knitwear|          A|      Ladieswear|             1|
Ladieswear|        11|    Womens Tailoring|            1003|
Knitwear|Jumper in a soft,...|NULL|  NULL|            ACTIVE|
NONE| 57|2c29ae653a9282cce...|
|00030bc026428965d...| 673677010|2019-11-28|0.0254067796610169|
2|      673677|      Henry polo (1)|            252|          Sweater|
Garment Upper body|               1010016|                       Solid|
43|         Dark Red|                          4|
Dark|                         18|                           Red|
1616|        Knitwear|          A|      Ladieswear|             1|
Ladieswear|        11|    Womens Tailoring|            1003|
Knitwear|Jumper in a soft,...|NULL|  NULL|            ACTIVE|
NONE| 57|2c29ae653a9282cce...|
|00083cda041544b2f...| 683001006|2019-02-10|0.0169322033898305|
2|      683001|    Pattern 7p Socks|            302|            Socks|
Socks & Tights|               1010001|         All over pattern|
10|            White|                          3|
Light|                          9|                          White|
3611|Shopbasket Socks|          B|Lingeries/Tights|             1|
Ladieswear|        62|Womens Nightwear,...|            1021|  Socks
and Tights|Fine-knit socks i...|NULL|  NULL|            ACTIVE|
NONE| 29|24e3594738f327e8a...|
|00083cda041544b2f...| 568571002|2018-10-24|0.0135423728813559|
2|      568571|Spartacus cheeky ...|             59|  Swimwear bottom|
Swimwear|               1010016|                   Solid|
9|            Black|                          4|
Dark|                          5|                          Black|
4242|        Swimwear|          B|Lingeries/Tights|             1|
Ladieswear|        60|Womens Swimwear, ...|            1018|
Swimwear|Fully lined, text...|NULL|  NULL|            ACTIVE|
NONE| 29|24e3594738f327e8a...|
|00083cda041544b2f...| 723469007|2019-02-10|0.0254067796610169|
2|      723469|Kelly Push (Melbo...|            306|              Bra|
Underwear|               1010016|                    Solid|
8|        Dark Grey|                          4|
Dark|                         12|                           Grey|
1334| Casual Lingerie|          B|Lingeries/Tights|             1|
Ladieswear|        61|     Womens Lingerie|            1017| Under-,
Nightwear|Push-up bras in s...|NULL|  NULL|            ACTIVE|
NONE| 29|24e3594738f327e8a...|
+--------------------+----------+----------+------------------
+---------------+------------+--------------------+--------------
+-----------------+------------------+----------------------
+-------------------------+-----------------+-----------------
+-------------------------+--------------------------
+-------------------------+----------------------------+-------------
+---------------+----------+----------------+--------------
+---------------+----------+--------------------+---------------
```

```
+------------------+--------------------+----+------
+------------------+----------------------+---+--------------------+
only showing top 5 rows
```

Limpieza y preparación

```
df_clean = df.select('customer_id', 'Fecha', 'price') \
    .where(col('customer_id').isNotNull() & col('Fecha').isNotNull() &
(col('price') > 0))
print(f"Registros limpios: {df_clean.count():,}")

Registros limpios: 31,788,324
```

Métricas por cliente: total gastado, número de compras y variabilidad mensual

```
clientes = df_clean.groupBy('customer_id') \
    .agg(
        spark_sum('price').alias('total_gastado'),
        count('price').alias('num_compras')
    )

gasto_mensual = df_clean.withColumn('año',
year('Fecha')).withColumn('mes', month('Fecha')) \
    .groupBy('customer_id', 'año', 'mes') \
    .agg(spark_sum('price').alias('gasto_mes'))

variabilidad = gasto_mensual.groupBy('customer_id') \
    .agg(stddev('gasto_mes').alias('std_gasto_mes'))

clientes = clientes.join(variabilidad, on='customer_id', how='left')
```

Segmentación ABC (por valor)

- A: Top 5% de clientes por gasto total (más valiosos)
- B: Siguiente 15%
- C: 80% restantes

```
clientes_pd = clientes.toPandas()
clientes_pd['ABC'] = pd.qcut(clientes_pd['total_gastado'], q=[0, .8,
.95, 1], labels=['C', 'B', 'A'])
```

Segmentación XYZ (por variabilidad)

- X: Clientes con gasto mensual más estable (menor desviación estándar)
- Y: Variabilidad intermedia
- Z: Clientes con gasto más variable

```
# Segmentación XYZ (por variabilidad)
std_values = clientes_pd['std_gasto_mes'].fillna(0)
bins = pd.qcut(std_values, q=[0, .33, .66, 1], retbins=True,
duplicates='drop')[1]
num_bins = len(bins) - 1

if num_bins == 3:
    etiquetas = ['Z', 'Y', 'X']
elif num_bins == 2:
    etiquetas = ['Z', 'X']
else:
    etiquetas = ['X']

clientes_pd['XYZ'] = pd.cut(std_values, bins=bins, labels=etiquetas,
include_lowest=True)
```

```
---------------------------------------------------------------------
-----
TypeError                                 Traceback (most recent call
last)
Cell In[7], line 13
     10 else:
     11     etiquetas = ['X']
---> 13 clientes_pd['XYZ'] = pd.qcut(std_values, q=bins,
labels=etiquetas, include_lowest=True)

TypeError: qcut() got an unexpected keyword argument 'include_lowest'
```

Visualización de segmentos ABC y XYZ

```
plt.figure(figsize=(8,6))
sns.countplot(x='ABC', data=clientes_pd, palette='Set2')
plt.title('Distribución de clientes por segmento ABC')
plt.xlabel('Segmento ABC')
plt.ylabel('Número de clientes')
plt.show()

plt.figure(figsize=(8,6))
sns.countplot(x='XYZ', data=clientes_pd, palette='Set1')
plt.title('Distribución de clientes por segmento XYZ')
plt.xlabel('Segmento XYZ')
plt.ylabel('Número de clientes')
plt.show()
```

Matriz combinada ABC/XYZ

```
plt.figure(figsize=(8,6))
sns.heatmap(pd.crosstab(clientes_pd['ABC'], clientes_pd['XYZ']),
annot=True, fmt='d', cmap='Blues')
```

```
plt.title('Matriz ABC/XYZ de clientes')
plt.xlabel('Segmento XYZ')
plt.ylabel('Segmento ABC')
plt.show()
```

Análisis profesional y recomendaciones

```
from IPython.display import Markdown
Markdown("""
**Análisis profesional:**

- Segmento A: Clientes que más aportan al negocio, deben ser prioridad en retención y recompensas.
- Segmento X: Clientes con compras estables, ideales para predicción y programas de lealtad.
- Segmento AX: Clientes más valiosos y predecibles, foco de campañas VIP.
- Segmentos CZ y BY: Clientes menos prioritarios o con comportamiento variable, útiles para campañas de activación.

**Recomendaciones:**
- Personalizar ofertas y comunicaciones para clientes AX.
- Incentivar la recurrencia en segmentos B y C.
- Analizar causas de variabilidad en segmentos Y y Z para mejorar la estabilidad.
- Monitorear la evolución de los segmentos para ajustar estrategias de marketing.
""")
```

Exportación de resultados y cierre de Spark

```
clientes_pd.to_csv('segmentacion_abc_xyz_hym.csv', index=False)
print('Segmentación exportada a segmentacion_abc_xyz_hym.csv')

spark.stop()
```